

Enumerators

- Often we will want to cycle through the elements in a collection.
- For example, you might want to display each element.
- One way to do this is to use a **foreach loop, as the preceding**
- examples have done.
- Another way is to use an enumerator.

- An *enumerator* is an object that implements either the non-generic **IEnumerator** or the generic **IEnumerator<T>** interface.
- **IEnumerator** defines one property called Current.
- The non-generic version is :
object Current { get; }
- For **IEnumerator<T>**, Current is declared like this:
- **T Current { get; }**

- In both cases, **Current** obtains the current element being enumerated.
- Since Current is a read-only property, an enumerator can only be used to retrieve, but not modify, the objects in a collection.

- **IEnumerator defines two methods.**
- The first is MoveNext():
- `bool MoveNext( )`

- Each call to **MoveNext()** moves the current position of the enumerator to the next element in the collection.
- It returns true if the next element is available, or false if the end of the collection has been reached.
- Prior to the first call to **MoveNext()**, the value of **Current** is undefined.
- (Conceptually, prior to the first call to **MoveNext()**, **the enumerator refers to the** nonexistent element that is just before the first element. Thus, you must call **MoveNext()** to move to the first element.)

- You can reset the enumerator to the start of the collection by calling **Reset()**, shown here:
void Reset()
- After calling **Reset()**, enumeration will again begin at the start of the collection.
- Thus, you must call **MoveNext()** before obtaining the first element.
- In **IEnumerator<T>**, the methods **MoveNext()** and **Reset()** work in the same way.

Two other points:

- First, you cannot use an enumerator to change the collection that it is enumerating.
- Thus, enumerators are read-only relative to the collection.
- Second, any change to the collection under enumeration invalidates the enumerator.

Using an Enumerator

- Before we can access a collection through an enumerator, we must obtain one.
- Each of the collection classes provides a **GetEnumerator()** method that returns an enumerator to the start of the collection.
- Using this enumerator, you can access each element in the collection, one element at a time.

- In general, to use an enumerator to cycle through the contents of a collection, follow these steps:
1. Obtain an enumerator to the start of the collection by calling the collection's **GetEnumerator()** method.
 2. Set up a loop that makes a call to **MoveNext()**. Have the loop iterate as long as **MoveNext()** returns true.
 3. Within the loop, obtain each element through **Current**.

- Here is an example that implements these steps.
- It uses an **ArrayList**, but the general principles apply to any type of collection, including the generic collections.

```
// Demonstrate an enumerator.  
using System;  
using System.Collections;  
class EnumeratorDemo {  
    static void Main() {  
        ArrayList list = new ArrayList(10);  
        for(int i=0; i < 10; i++)  
            list.Add(i);  
        // Use enumerator to access list.  
        IEnumerator etr = list.GetEnumerator();  
        while(etr.MoveNext())  
            Console.Write(etr.Current + " ");  
        Console.WriteLine();  
        // Re-enumerate the list.  
        etr.Reset();  
        while(etr.MoveNext())  
            Console.Write(etr.Current + " ");  
        Console.WriteLine();  
    }  
}
```

- The output is shown here:

0 1 2 3 4 5 6 7 8 9

0 1 2 3 4 5 6 7 8 9

- In general, when you need to cycle through a collection, a **foreach loop** is more convenient to use than an enumerator.
- However, an enumerator gives you a little extra control by allowing you to reset the enumerator at will.

Using the IDictionaryEnumerator

- When using a non-generic **IDictionary**, such as Hashtable, you will use an **IDictionaryEnumerator** instead of an **IEnumerator** when cycling through the collection.

- The **IDictionaryEnumerator** inherits **IEnumerator** and adds three properties.
- The first is DictionaryEntry Entry { get; }
- **Entry obtains the next key/value pair from the enumerator in the form of a DictionaryEntry structure.**

- Recall that **DictionaryEntry** defines two **properties**, called **Key** and **Value**, which can be used to access the key or value contained within the entry.
- The other two properties defined by **IDictionaryEnumerator** are shown here:
 object Key { get; }
 object Value { get; }
- These allow you to access the key or value directly.

- An **IDictionaryEnumerator** is used just like a **regular enumerator**, except that you will obtain the current value through the **Entry, Key, or Value properties** rather than **Current**.
- Thus, after obtaining an **IDictionaryEnumerator**, **you must call MoveNext()** to obtain the first element.
- Continue to call **MoveNext()** to obtain the rest of the elements in the collection.
- **MoveNext()** returns false when there are no more elements.

- Here is an example that enumerates the elements in a **Hashtable** through an **IDictionaryEnumerator**:

```
// Demonstrate IDictionaryEnumerator.
using System;
using System.Collections;
class IDicEnumDemo {
static void Main() {
// Create a hash table.
Hashtable ht = new Hashtable();
// Add elements to the table.
ht.Add("Tom", "555-3456");
ht.Add("Mary", "555-9876");
ht.Add("Todd", "555-3452");
ht.Add("Ken", "555-7756");
// Demonstrate enumerator.
IDictionaryEnumerator etr = ht.GetEnumerator();
Console.WriteLine("Display info using Entry.");
while(etr.MoveNext())
Console.WriteLine(etr.Entry.Key + ": " +
etr.Entry.Value);
Console.WriteLine();
Console.WriteLine("Display info using Key and Value directly.");
etr.Reset();
while(etr.MoveNext())
Console.WriteLine(etr.Key + ": " +
etr.Value);
}
}
```

- The output is shown here:

Display info using Entry.

Ken: 555–7756

Mary: 555–9876

Tom: 555–3456

Todd: 555–3452

Display info using Key and Value directly.

Ken: 555–7756

Mary: 555–9876

Tom: 555–3456

Todd: 555–3452

Implementing IEnumerable and IEnumerator

- As mentioned earlier, normally it is easier (and better) to use a **foreach loop to cycle through** a collection than it is to explicitly use **IEnumerator methods**.
- **However, understanding the operation of these interfaces is important for another reason:**
- If you want to create a class that contains objects that can be enumerated via a **foreach loop, then that class must implement IEnumerator**.
- **It must also implement IEnumerable.**
- **In other words, to enable an object of a class that you create to be used in a foreach loop, you must implement IEnumerator and**
- **IEnumerable, using either their generic or non-generic form.**
- **Fortunately, because these interfaces are so small, they are easy to implement.**

- Here is an example that implements the non-generic versions of **IEnumerable** and **IEnumerator** so that the contents of the array encapsulated within **MyClass** can be Enumerated:

```
// Implement IEnumerable and IEnumerator.
using System;
using System.Collections;
class MyClass : IEnumerator, IEnumerable {
    char[] chrs = { 'A', 'B', 'C', 'D' };
    int idx = -1;
    // Implement IEnumerable.
    public IEnumerator GetEnumerator() {
        return this;
    }
    // The following methods implement IEnumerator.
    // Return the current object.
    public object Current {
        get {
            return chrs[idx];
        }
    }
}
```

```
// Advance to the next object.  
public bool MoveNext() {  
    if(idx == chrs.Length-1) {  
        Reset(); // reset enumerator at the end  
        return false;  
    }  
    idx++;  
    return true;  
}  
// Reset the enumerator to the start.  
public void Reset() { idx = -1; }  
}
```

```
class EnumeratorImplDemo {  
    static void Main() {  
        MyClass mc = new MyClass();  
        // Display the contents of mc.  
        foreach(char ch in mc)  
            Console.Write(ch + " ");  
        Console.WriteLine();  
        // Display the contents of mc, again.  
        foreach(char ch in mc)  
            Console.Write(ch + " ");  
        Console.WriteLine();  
    }  
}
```

- Here is the output:

A B C D

A B C D

Iterators

- As the preceding example shows, it is not difficult to implement **IEnumerator** and **IEnumerable**.
- However, it can be made even easier through the use of an *iterator*.

- ***An iterator*** is a method, operator, or accessor that returns the members of a set of objects, one member at a time, from start to finish.
- For example, assuming some array that has five elements, then an iterator for that array will return those five elements, one at a time.

- By implementing an iterator, you make it possible for an object of a class to be used in a **foreach loop**.
- Let's begin with an example of a simple iterator.
- The following program is a modified version of the preceding program that uses an iterator rather than explicitly implementing **IEnumerator** and **IEnumerable**.

```
// A simple example of an iterator.
using System;
using System.Collections;
class MyClass {
char[] chrs = { 'A', 'B', 'C', 'D' };
// This iterator returns the characters
// in the chrs array.
public IEnumerator GetEnumerator() {
foreach(char ch in chrs)
yield return ch;
}
}
class ItrDemo {
static void Main() {
MyClass mc = new MyClass();
foreach(char ch in mc)
Console.Write(ch + " ");
Console.WriteLine();
}
}
```

- The output is shown here:

A B C D

- As you can see, the contents of mc.chrs were enumerated.
- First, notice that MyClass does not specify IEnumerator as an implemented interface.
- When creating an iterator, the compiler automatically implements this interface.
- Second, notice the GetEnumerator() method, which is shown again here for your convenience

- // This iterator returns the characters
- // in the chrs array.

```
public IEnumerator GetEnumerator() {  
    foreach(char ch in chrs)  
        yield return ch;  
}
```

- This is the iterator for MyClass.
- Notice that it implicitly implements the GetEnumerator() method defined by IEnumerable.
- Consider the body of the method.
- It contains a foreach loop that returns the elements in chrs.
- It does this through the use of a yield return statement.
- The yield return statement returns the next object in the collection, which in this case is the next character in chrs.
- This feature enables mc (a MyClass object) to be used within the foreach loop inside Main().

- The term **yield** is a *contextual keyword in the C# language*.
- *This means that it only has* special meaning inside an iterator block.
- Outside of an iterator, **yield** can be used like any other identifier.

- One important point to understand is that an iterator does not need to be backed by an array or other type of collection.
- It simply must return the next element in a group of elements.
- This means the elements can be dynamically constructed using an algorithm.
- For example, here is a version of the previous program that returns all uppercase letters in the alphabet.
- Instead of using an array, it generates the letters using a for loop.

```
// Iterated values can be dynamically constructed.
using System;
using System.Collections;
class MyClass {
    char ch = 'A';
    // This iterator returns the letters of the alphabet.
    public IEnumerator GetEnumerator() {
        for(int i=0; i < 26; i++)
            yield return (char) (ch + i);
    }
}

class ItrDemo2 {
    static void Main() {
        MyClass mc = new MyClass();
        foreach(char ch in mc)
            Console.Write(ch + " ");
        Console.WriteLine();
    }
}
```

- The output is shown here:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Stopping an Iterator

- You can stop an iterator early by using this form of the yield statement:
yield break;
- When this statement executes, the iterator signals that the end of the collection has been reached, which effectively stops the iterator.
- The following program modifies the preceding program so that it displays only the first ten letters in the alphabet.

```
// Use yield break.  
using System;  
using System.Collections;  
class MyClass {  
    char ch = 'A';  
    // This iterator returns the first 10  
    // letters of the alphabet.  
    public IEnumerator GetEnumerator() {  
        for(int i=0; i < 26; i++) {  
            if(i == 10) yield break; // stop iterator early  
            yield return (char) (ch + i);  
        }  
    }  
}
```

```
class ItrDemo3 {  
    static void Main() {  
        MyClass mc = new MyClass();  
        foreach(char ch in mc)  
            Console.Write(ch + " ");  
        Console.WriteLine();  
    }  
}
```

- The output is shown here:

A B C D E F G H I J

Using Multiple yield Directives

- You can have more than one yield statement in an iterator.
- However, each yield must return the next element in the collection.
- For example, Next slide

```
// Multiple yield statements are allowed.
using System;
using System.Collections;
class MyClass {
// This iterator returns the letters
// A, B, C, D, and E.
public IEnumerator GetEnumerator() {
yield return 'A';
yield return 'B';
yield return 'C';
yield return 'D';
yield return 'E';
}
}
```

- class ItrDemo5 {
- static void Main() {
- MyClass mc = new MyClass();
- foreach(char ch in mc)
- Console.Write(ch + " ");
- Console.WriteLine();
- }
- }

- The output is shown here:

A B C D E

- Inside GetEnumerator(), five yield statements occur.
- The important thing to understand is that they are executed one at a time, in order, each time another element in the collection is obtained.
- Thus, each time through the foreach loop in Main(), one character is returned.

Creating a Named Iterator

- Although the preceding examples have shown the easiest way to implement an iterator, there is an alternative:
 - the named iterator.
- In this approach, you create a method, operator, or accessor that returns a reference to an **IEnumerable object**.
- The code will use this object to supply the iterator.
- A named iterator is a method with the following general form:

```
public IEnumerable itr-name(param-list) {  
    // ...  
    yield return obj;  
}
```

- Here, itr-name is the name of the method, param-list specifies zero or more parameters that will be passed to the iterator method, and obj is the next object returned by the iterator.
- Once we have created a named iterator, we can use it anywhere that an iterator is needed.
- For example, we can use the named iterator to control a foreach loop.

- Named iterators are very useful in some circumstances because they allow you to pass arguments to the iterator that control what elements are obtained.
- For example, you might pass the iterator the beginning and ending points of a range of elements to iterate.
- This form of iterator can also be overloaded, further adding to its flexibility.

- The following program illustrates two ways that a named iterator can be used to obtain elements.
- One enumerates a range of elements given the endpoints.
- The other enumerates the elements beginning at the start of the sequence and ending at the specified stopping point.

```
// Use named iterators.
using System;
using System.Collections;
class MyClass {
    char ch = 'A';
    // This iterator returns the letters
    // of the alphabet, beginning at A and
    // stopping at the specified stopping point.
    public IEnumerable Myltr(int end) {
        for(int i=0; i < end; i++)
            yield return (char) (ch + i);
    }
    // This iterator returns the specified
    // range of letters.
    public IEnumerable Myltr(int begin, int end) {
        for(int i=begin; i < end; i++)
            yield return (char) (ch + i);
    }
}
```

```
class ItrDemo4 {  
    static void Main() {  
        MyClass mc = new MyClass();  
        Console.WriteLine("Iterate the first 7 letters:");  
        foreach(char ch in mc.Myltr(7))  
            Console.Write(ch + " ");  
        Console.WriteLine("\n");  
        Console.WriteLine("Iterate letters from F to L:");  
        foreach(char ch in mc.Myltr(5, 12))  
            Console.Write(ch + " ");  
        Console.WriteLine();  
    }  
}
```

- The output is shown here:

Iterate the first 7 letters:

A B C D E F G

Iterate letters from F to L:

F G H I J K L

Creating a Generic Iterator

- The preceding examples of iterators have been non-generic, but it is, of course, also possible to create generic iterators.
- Doing so is quite easy: Simply return an object of the generic `IEnumerator<T>` or `IEnumerable<T>` type.
- Here is an example that creates a generic iterator:

```
// A simple example of a generic iterator.  
using System;  
using System.Collections.Generic;  
class MyClass<T> {  
    T[] array;  
    public MyClass(T[] a) {  
        array = a;  
    }  
    // This iterator returns the characters  
    // in the chrs array.  
    public IEnumerator<T> GetEnumerator() {  
        foreach(T obj in array)  
            yield return obj;  
    }  
}
```

```
class GenericItrDemo {  
    static void Main() {  
        int[] nums = { 4, 3, 6, 4, 7, 9 };  
        MyClass<int> mc = new MyClass<int>(nums);  
        foreach(int x in mc)  
            Console.Write(x + " ");  
        Console.WriteLine();  
        bool[] bVals = { true, true, false, true };  
        MyClass<bool> mc2 = new MyClass<bool>(bVals);  
        foreach(bool b in mc2)  
            Console.Write(b + " ");  
        Console.WriteLine();  
    }  
}
```

- The output is shown here:

4 3 6 4 7 9

True True False True

- In this example, the array containing the objects to be iterated is passed to MyClass through its constructor.
- The type of the array is specified as a type argument to MyClass.
- The **GetEnumerator()** method operates on data of type T and returns an **IEnumerator<T>** enumerator.
- Thus, the iterator defined by **MyClass** can **enumerate any type of data.**

